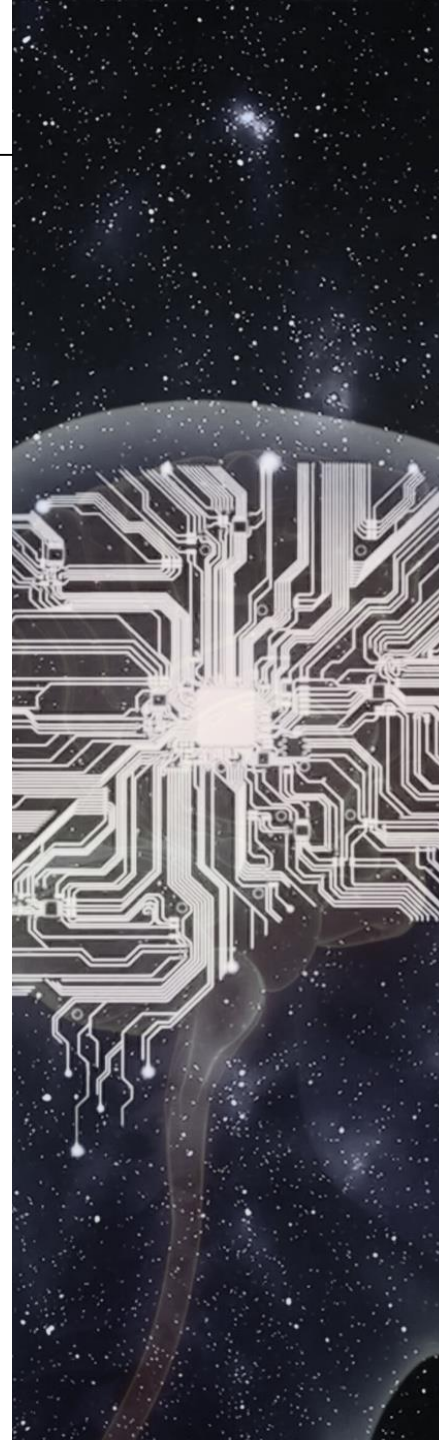


Assembly Programming

4. Data Transfers, Addressing, and Arithmetic



Operand Types

```
[label:] mnemonic [operands][ ; comment ]
```

mnemonic

mnemonic [*destination*]

mnemonic [*destination*], [*source*]

mnemonic [*destination*], [*source-1*], [*source-2*]

- Immediate—uses a numeric literal expression
- Register—uses a named register in the CPU
- Memory—references a memory location

Data Transfer Instructions

Table 4-1 Instruction Operand Notation, 32-Bit Mode.

Operand	Description
<i>reg8</i>	8-bit general-purpose register: AH, AL, BH, BL, CH, CL, DH, DL
<i>reg16</i>	16-bit general-purpose register: AX, BX, CX, DX, SI, DI, SP, BP
<i>reg32</i>	32-bit general-purpose register: EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP
<i>reg</i>	Any general-purpose register
<i>sreg</i>	16-bit segment register: CS, DS, SS, ES, FS, GS
<i>imm</i>	8-, 16-, or 32-bit immediate value
<i>imm8</i>	8-bit immediate byte value
<i>imm16</i>	16-bit immediate word value
<i>imm32</i>	32-bit immediate doubleword value
<i>reg/mem8</i>	8-bit operand, which can be an 8-bit general register or memory byte
<i>reg/mem16</i>	16-bit operand, which can be a 16-bit general register or memory word
<i>reg/mem32</i>	32-bit operand, which can be a 32-bit general register or memory doubleword
<i>mem</i>	An 8-, 16-, or 32-bit memory operand

Direct Memory Operands

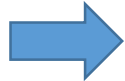
Variable names are references to offsets within the data segment.

```
.data
```

```
var1 BYTE 10h
```



```
mov al var1
```



```
A0 00010400
```

Suppose var1 were located at offset 10400h



MOV Instruction

The MOV instruction copies data from a source operand to a destination operand.

`MOV destination, source`  `dest = source;`

- Both operands must be the same size.
- Both operands cannot be memory operands.
- The instruction pointer register (IP, EIP, or RIP) cannot be a destination operand.

```
MOV reg, reg
MOV mem, reg
MOV reg, mem
MOV mem, imm
MOV reg, imm
```

```
.data
var1 WORD ?
var2 WORD ?
.code
mov ax, var1
mov var2, ax
```

Memory to Memory

A single MOV instruction cannot be used to move data directly from one memory location to another.

Overlapping Values

.data

oneByte BYTE 78h

oneWord WORD 1234h

oneDword DWORD 12345678h

.code

mov eax, 0 ; EAX = 00000000h

mov al, oneByte ; EAX = 00000078h

mov ax, oneWord ; EAX = 00001234h

mov eax, oneDword ; EAX = 12345678h

mov ax, 0 ; EAX = 12340000h

Zero/Sign Extension of Integers

Copying Smaller Values to Larger Ones

```
.data  
count WORD 1  
.code  
mov ecx,0  
mov cx,count
```

```
.data  
signedVal SWORD -16 ; FFF0h (-16)  
.code  
mov ecx,0  
mov cx,signedVal ; ECX = 0000FFF0h (+65,520)
```

```
mov ecx,0FFFFFFFFh  
mov cx,signedVal ; ECX = FFFFFFFF0h (-16)
```

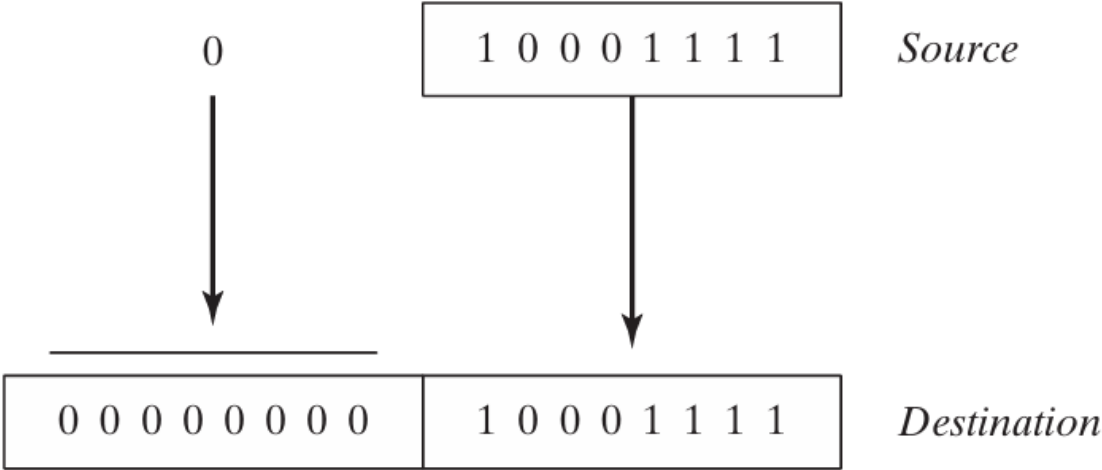
MOVZX Instruction

The MOVZX instruction (move with zero-extend) copies the contents of a source operand into a destination operand and zero-extends the value to 16 or 32 bits.

```
MOVZX    reg32, reg/mem8
MOVZX    reg32, reg/mem16
MOVZX    reg16, reg/mem8

.data
byteVal  BYTE 10001111b
.code
movzx    ax, byteVal                ; AX = 0000000010001111b
```

FIGURE 4–1 Using MOVZX to copy a byte into a 16-bit destination.



```
mov      bx, 0A69Bh
movzx    eax, bx                ; EAX = 0000A69Bh
movzx    edx, bl                ; EDX = 0000009Bh
movzx    cx, bl                 ; CX = 009Bh

.data
byte1    BYTE 9Bh
word1    WORD 0A69Bh
.code
movzx    eax, word1            ; EAX = 0000A69Bh
movzx    edx, byte1            ; EDX = 0000009Bh
movzx    cx, byte1             ; CX = 009Bh
```

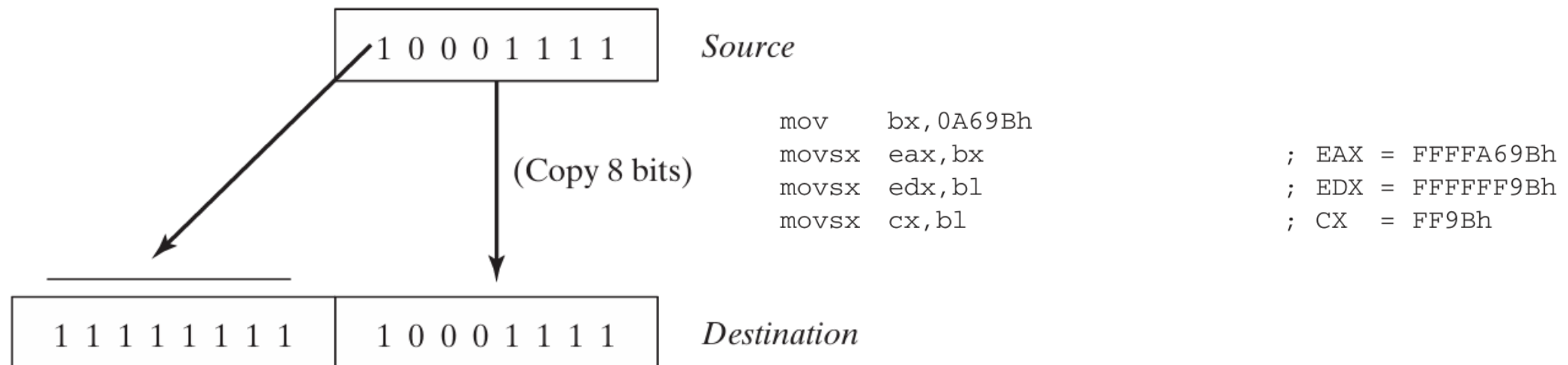
Data Transfer Instructions

MOVSX Instruction

The MOVSX instruction (move with sign-extend) copies the contents of a source operand into a destination operand and sign-extends the value to 16 or 32 bits.

```
MOVSX  reg32,reg/mem8      .data
MOVSX  reg32,reg/mem16     byteVal BYTE 10001111b
MOVSX  reg16,reg/mem8      .code
                           movsx  ax,byteVal                ; AX = 1111111110001111b
```

FIGURE 4–2 Using MOVSX to copy a byte into a 16-bit destination.



LAHF and SAHF Instructions

The LAHF (load status flags into AH) instruction copies the low byte of the EFLAGS register into AH.

```
.data
saveflags BYTE ?
.code
lahf                ; load flags into AH
mov  saveflags,ah   ; save them in a variable
```

The SAHF (store AH into status flags) instruction copies AH into the low byte of the EFLAGS (or RFLAGS) register.

```
mov  ah,saveflags   ; load saved flags into AH
sahf                ; copy into Flags register
```

XCHG Instruction

The XCHG (exchange data) instruction exchanges the contents of two operands.

XCHG *reg, reg*

XCHG *reg, mem*

XCHG *mem, reg*

xchg ax, bx ; exchange 16-bit regs

xchg ah, al ; exchange 8-bit regs

xchg var1, bx ; exchange 16-bit mem op with BX

xchg eax, ebx ; exchange 32-bit regs

mov ax, val1

xchg ax, val2

mov val1, ax

To exchange two memory operands

Direct-Offset Operands

access memory locations that may not have explicit labels.

```
arrayB  BYTE 10h,20h,30h,40h,50h
```

```
mov al,arrayB           ; AL = 10h
mov al,[arrayB+1]       ; AL = 20h
mov al,[arrayB+2]       ; AL = 30h
```

```
.data
arrayW WORD 100h,200h,300h
```

```
.code
mov ax,arrayW           ; AX = 100h
mov ax,[arrayW+2]       ; AX = 200h
```

```
.data
arrayD DWORD 10000h,20000h
```

```
.code
mov eax,arrayD          ; EAX = 10000h
mov eax,[arrayD+4]      ; EAX = 20000h
```

Data Transfer Instructions

Example Program (Moves)

```
.data
val1 WORD 1000h
val2 WORD 2000h

arrayB BYTE 10h,20h,30h,40h,50h
arrayW WORD 100h,200h,300h
arrayD DWORD 10000h,20000h

.code
main proc

; MOVZX
    mov     bx,0A69Bh
    movzx   eax,bx           ; EAX = 0000A69Bh
    movzx   edx,bl           ; EDX = 0000009Bh
    movzx   cx,bl            ; CX = 009Bh

; MOVSX
    mov     bx,0A69Bh
    movsx   eax,bx           ; EAX = FFFFA69Bh
    movsx   edx,bl           ; EDX = FFFFFFF9Bh
    mov     bl,7Bh
    movsx   cx,bl            ; CX = 007Bh
```

```
; Memory-to-memory exchange:
    mov     ax,val1          ; AX = 1000h
    xchg    ax,val2          ; AX = 2000h, val2 = 1000h
    mov     val1,ax          ; val1 = 2000h

; Direct-Offset Addressing (byte array):
    mov     al,arrayB        ; AL = 10h
    mov     al,[arrayB+1]    ; AL = 20h
    mov     al,[arrayB+2]    ; AL = 30h

; Direct-Offset Addressing (word array):
    mov     ax,arrayW        ; AX = 100h
    mov     ax,[arrayW+2]    ; AX = 200h

; Direct-Offset Addressing (doubleword array):
    mov     eax,arrayD
    ; EAX = 10000h
    mov     eax,[arrayD+4]    ; EAX = 20000h
    mov     eax,[arrayD+TYPE arrayD] ; EAX = 20000h

    Invoke ExitProcess,0

main endp
end main
```

Data Transfer Instructions

INC and DEC Instructions

The INC (increment) and DEC (decrement) instructions, respectively, add 1 and subtract 1 from a register or memory operand.

```
INC reg/mem
```

```
DEC reg/mem
```

```
.data
```

```
myWord WORD 1000h
```

```
.code
```

```
inc myWord ; myWord = 1001h
```

```
mov bx,myWord
```

```
dec bx ; BX = 1000h
```

ADD Instruction

The ADD instruction adds a source operand to a destination operand of the same size.

ADD dest, source

Source is unchanged by the operation, and the sum is stored in the destination operand.

```
.data
var1 DWORD 10000h
var2 DWORD 20000h
.code
mov  eax, var1           ; EAX = 10000h
add  eax, var2           ; EAX = 30000h
```

Flags

The Carry, Zero, Sign, Overflow, Auxiliary Carry, and Parity flags are changed according to the value that is placed in the destination operand.

NEG Instruction

The NEG (negate) instruction reverses the sign of a number by converting the number to its two's complement.

NEG *reg*

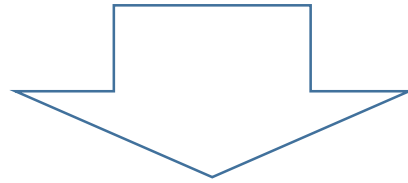
NEG *mem*

Flags

The Carry, Zero, Sign, Overflow, Auxiliary Carry, and Parity flags are changed according to the value that is placed in the destination operand.

Implementing Arithmetic Expressions

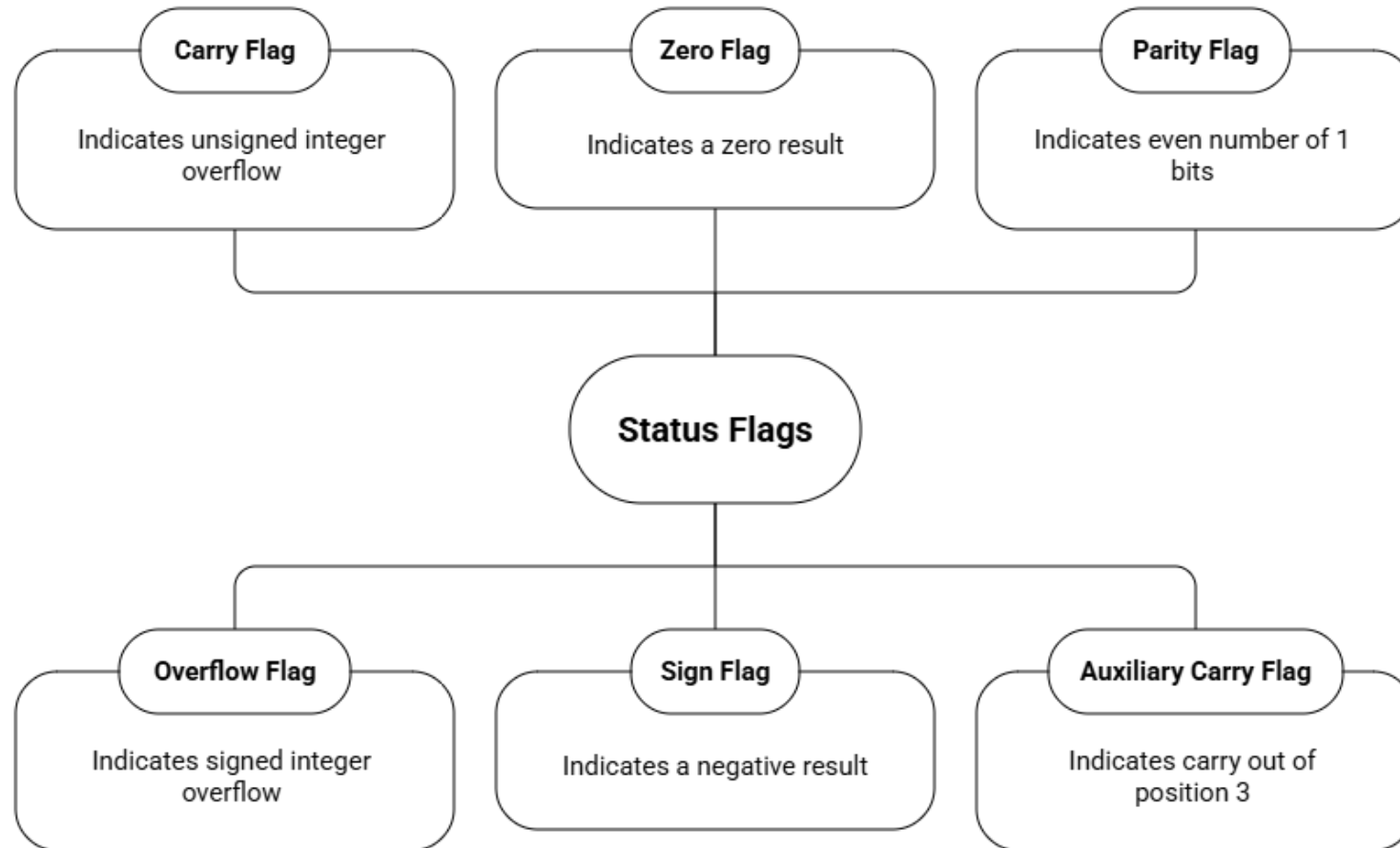
`Rval = -Xval + (Yval - Zval);`



<code>Rval SDWORD ?</code>		<code>; first term: -Xval</code>	
<code>Xval SDWORD 26</code>		<code>mov eax,Xval</code>	
<code>Yval SDWORD 30</code>		<code>neg eax</code>	<code>; EAX = -26</code>
<code>Zval SDWORD 40</code>			
		<code>; second term: (Yval - Zval)</code>	
		<code>mov ebx,Yval</code>	
		<code>sub ebx,Zval</code>	<code>; EBX = -10</code>
		<code>; add the terms and store:</code>	
		<code>add eax,ebx</code>	
		<code>mov Rval,eax</code>	<code>; -36</code>

Flags Affected by Addition and Subtraction

Understanding Status Flags



Unsigned Operations: Zero, Carry, and Auxiliary Carry

The Zero flag is set when the result of an arithmetic operation equals zero.

```
mov  ecx,1
sub  ecx,1                ; ECX = 0, ZF = 1
mov  eax,0FFFFFFFFh
inc  eax                  ; EAX = 0, ZF = 1
inc  eax                  ; EAX = 1, ZF = 0
dec  eax                  ; EAX = 0, ZF = 1
```

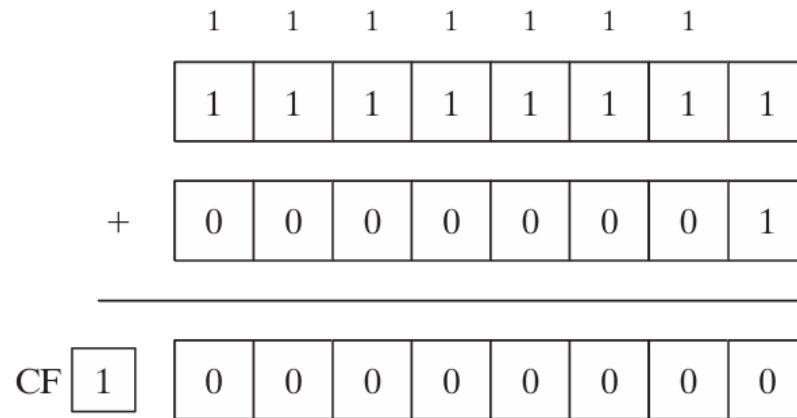
The INC and DEC instructions do not affect the Carry flag. Applying the NEG instruction to a nonzero operand always sets the Carry flag.

Addition and Subtraction

Addition and the Carry Flag

```
mov  al,0FFh  
add  al,1                                ; AL = 00, CF = 1
```

FIGURE 4-3 Adding 1 to 0FFh sets the Carry flag.



```
mov  ax,00FFh  
add  ax,1                                ; AX = 0100h, CF = 0  
  
mov  ax,0FFFFh  
add  ax,1                                ; AX = 0000, CF = 1
```

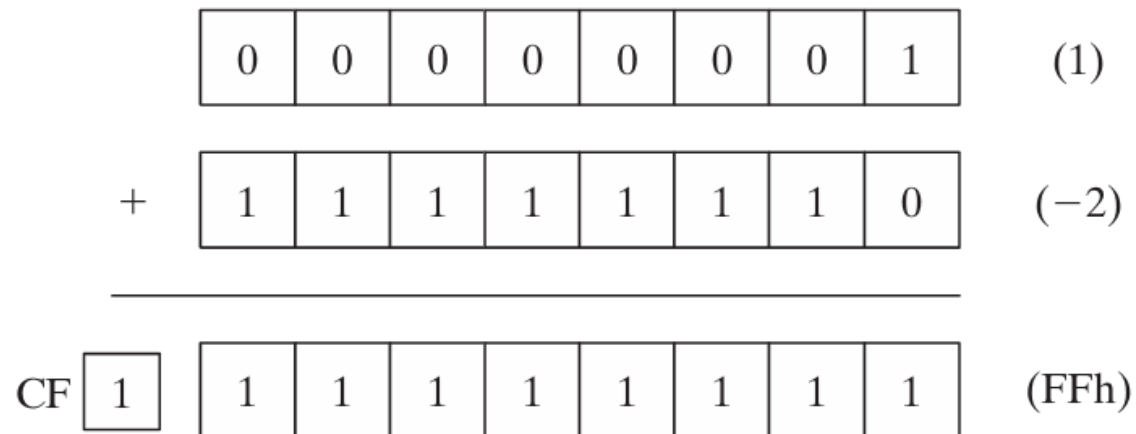
Addition and Subtraction

Subtraction and the Carry Flag

A subtract operation sets the Carry flag when a larger unsigned integer is subtracted from a smaller one.

```
mov  al,1
sub  al,2                ; AL = FFh, CF = 1
```

FIGURE 4-4 Subtracting 2 from 1 sets the Carry flag.



```
; 자리(범위)를 넘어간 경우
mov al,0FFh
add al,1 ; CF = 1, AL = 00

; 0 이하의 값으로 연산되는 경우
mov al,0
sub al,1 ; CF = 1, AL = FF
```

Addition and Subtraction

Auxiliary Carry

The Auxiliary Carry (AC) flag indicates a carry or borrow out of bit 3 in the destination operand.

```
mov  al,0Fh
```

```
add  al,1                                ; AC = 1
```

```
  0 0 0 0 1 1 1 1
+ 0 0 0 0 0 0 0 1
-----
  0 0 0 1 0 0 0 0
```

Parity

The Parity flag (PF) is set when the least significant byte of the destination has an even number of 1 bits.

```
mov  al,10001100b
```

```
add  al,00000010b                        ; AL = 10001110, PF = 1
```

```
sub  al,10000000b                        ; AL = 00001110, PF = 0
```

Signed Operations: Sign and Overflow Flags

Sign Flag

The Sign flag is set when the result of a signed arithmetic operation is negative.

```
mov  eax, 4
sub  eax, 5                ; EAX = -1, SF = 1

mov  bl, 1                ; BL = 01h
sub  bl, 2                ; BL = FFh (-1), SF = 1
```

Overflow Flag

The Overflow flag is set when the result of a signed arithmetic operation overflows or underflows the destination operand.

```
mov  al, +127
add  al, 1                ; OF = 1

mov  al, -128
sub  al, 1                ; OF = 1
```

Addition and Subtraction

The Addition Test

- Adding two positive operands generates a negative sum
- Adding two negative operands generates a positive sum

How the Hardware Detects Overflow

The value that carries out of the highest bit position is exclusive ORed with the carry into the high bit of the result. The resulting value is placed in the Overflow flag.

FIGURE 4–5 Demonstration of how the Overflow flag is set.

		1	0	0	0	0	0	0
	+	1	1	1	1	1	1	0
CF	1	0	1	1	1	1	1	0

Addition and Subtraction

NEG Instruction

The NEG instruction produces an invalid result if the destination operand can not be stored correctly.

<code>mov al,-128</code>	<code>; AL = 10000000b</code>
<code>neg al</code>	<code>; AL = 10000000b, OF = 1</code>
<code>mov al,+127</code>	<code>; AL = 01111111b</code>
<code>neg al</code>	<code>; AL = 10000001b, OF = 0</code>

Addition and Subtraction

Example Program (*AddSubTest*)

; Addition and Subtraction (AddSubTest.asm)

; Chapter 4 example. Demonstration of ADD, SUB,
; INC, DEC, and NEG instructions, and how
; they affect the CPU status flags.

.386

.model flat,stdcall

.stack 4096

ExitProcess proto,dwExitCode:dword

.data

Rval SDWORD ?

Xval SDWORD 26

Yval SDWORD 30

Zval SDWORD 40

.code

main proc

; INC and DEC

mov ax,1000h

inc ax ; 1001h

dec ax ; 1000h

; Expression: Rval = -Xval + (Yval - Zval)

mov eax,Xval

neg eax ; -26

mov ebx,Yval

sub ebx,Zval ; -10

add eax,ebx

mov Rval,eax ; -36

; Zero flag example:

mov cx,1

sub cx,1 ; ZF = 1

mov ax,0FFFFh

inc ax ; ZF = 1

; Sign flag example:

mov cx,0

sub cx,1 ; SF = 1

mov ax,7FFFh

add ax,2 ; SF = 1

; Carry flag example:

mov al,0FFh

add al,1 ; CF = 1, AL = 00

Addition and Subtraction

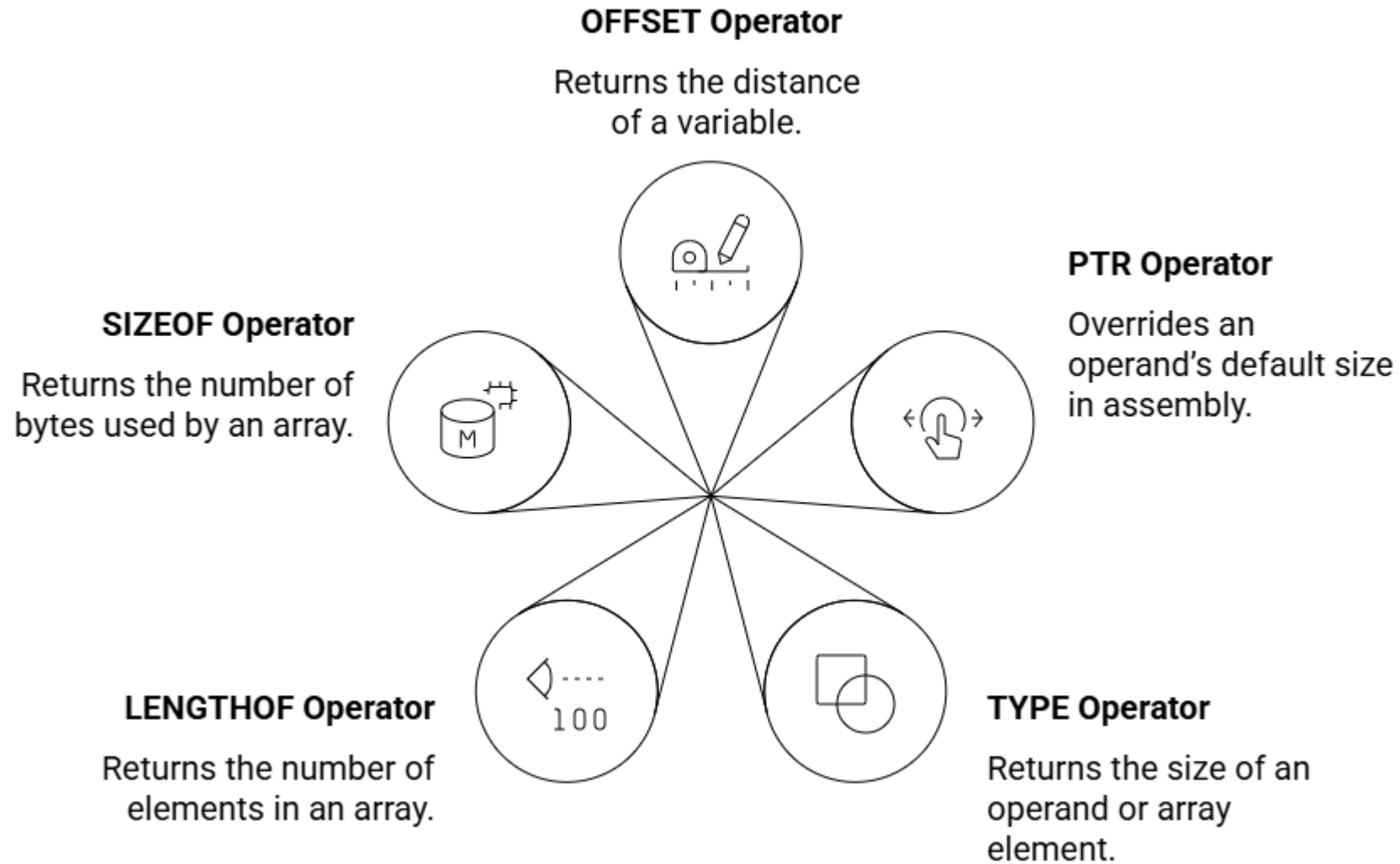
Example Program (*AddSubTest*)

```
    ; Overflow flag example:
    mov     al,+127
    add     al,1                ; OF = 1
    mov     al,-128
    sub     al,1                ; OF = 1

    invoke  ExitProcess,0
main endp
end main
```

Addition and Subtraction

Data-Related Operators and Directives

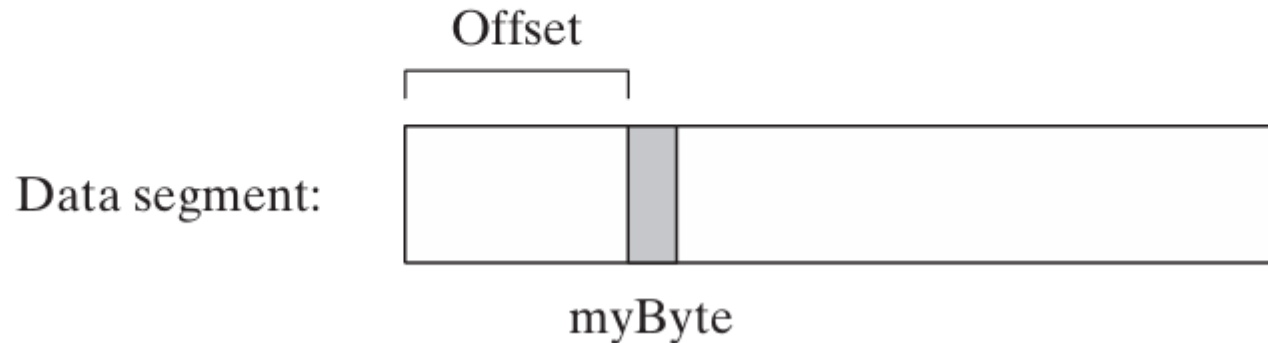


OFFSET Operator

The OFFSET operator returns the offset of a data label.

The offset represents the distance, in bytes, of the label from the beginning of the data segment.

FIGURE 4–6 A variable named myByte.



OFFSET Examples

<pre>.data bVal BYTE ? wVal WORD ? dVal DWORD ? dVal2 DWORD ?</pre>	<pre>mov esi,OFFSET bVal ; ESI = 00404000h mov esi,OFFSET wVal ; ESI = 00404001h mov esi,OFFSET dVal ; ESI = 00404003h mov esi,OFFSET dVal2 ; ESI = 00404007h</pre>
--	--

```
.data
myArray WORD 1,2,3,4,5
.code
mov esi,OFFSET myArray + 4  ESI points to the third integer in the array
```

```
.data
bigArray DWORD 500 DUP(?)
pArray DWORD bigArray  pArray points to the beginning of bigArray
```

ALIGN Directive

The ALIGN directive aligns a variable on a byte, word, doubleword, or paragraph boundary.

ALIGN bound

```
bVal  BYTE  ?           ; 00404000h
ALIGN 2
wVal  WORD  ?           ; 00404002h
bVal2 BYTE  ?           ; 00404004h
ALIGN 4
dVal  DWORD ?           ; 00404008h
dVal2 DWORD ?           ; 0040400Ch
```

Why bother aligning data? Because the CPU can process data stored at even numbered addresses more quickly than those at odd-numbered addresses.

PTR Operator

PTR operator to override the declared size of an operand.

```
.data
myDouble  DWORD  12345678h
.code
mov  ax,myDouble                ; error ➡ mov  ax,WORD PTR myDouble
```

FIGURE 4–7 Memory layout of myDouble.

Doubleword	Word	Byte	Offset	
12345678	5678	78	0000	myDouble
		56	0001	myDouble + 1
	1234	34	0002	myDouble + 2
		12	0003	myDouble + 3

```
mov  bl,BYTE PTR myDouble      ; 78h
```

```
.data
myDouble  DWORD  12346578h

.code
mov  ax, myDouble              ; size mismatch
mov  ax, WORD PTR myDouble     ; loads 5678h
mov  ax, WORD PTR [myDouble+2] ; loads 1234h
mov  WORD PTR myDouble, 4321h ; saves 4321h
```

```
myDouble  DWORD  12345678h

.code
mov  al, BYTE PTR  myDouble    ; AL = 78h
mov  al, BYTE PTR [myDouble + 1] ; AL = 56h
mov  al, BYTE PTR [myDouble + 2] ; AL = 34h
mov  ax, WORD PTR  myDouble    ; AX = 5678h
mov  ax, WORD PTR [myDouble + 1] ; AX = 1234h
```

TYPE Operator

The TYPE operator returns the size, in bytes, of a single element of a variable.

```
.data  
var1 BYTE ?  
var2 WORD ?  
var3 DWORD ?  
var4 QWORD ?
```

Expression	Value
TYPE var1	1
TYPE var2	2
TYPE var3	4
TYPE var4	8

LENGTHOF Operator

The LENGTHOF operator counts the number of elements in an array, defined by the values appearing on the same line as its label.

```
.data
byte1    BYTE    10,20,30
array1    WORD    30 DUP(??),0,0
array2    WORD    5 DUP(3 DUP(??))
array3    DWORD   1,2,3,4
digitStr  BYTE    "12345678",0
```

Expression	Value
LENGTHOF byte1	3
LENGTHOF array1	30 + 2
LENGTHOF array2	5 * 3
LENGTHOF array3	4
LENGTHOF digitStr	9

SIZEOF Operator

The SIZEOF operator returns a value that is equivalent to multiplying LENGTHOF by TYPE.

```
.data
intArray WORD 32 DUP(0)
.code
mov  eax,SIZEOF intArray      ; EAX = 64
```

LABEL Directive

The LABEL directive lets you insert a label and give it a size attribute without allocating any storage.

```
.data
val16 LABEL WORD
val32 DWORD 12345678h
.code
mov  ax,val16
mov  dx,[val16+2]
```

```
.data
LongValue LABEL DWORD
val1  WORD  5678h
val2  WORD  1234h
.code
mov  eax,LongValue      ; EAX = 12345678h
; AX = 5678h
; DX = 1234h
```


Indirect Operands

```
.data
byteVal BYTE 10h
.code
mov esi,OFFSET byteVal
mov al,[esi]                ; AL = 10h

inc [esi]                   ; error: operand must have size
  ↓
inc BYTE PTR [esi]
```

Indirect Addressing

Arrays

```
.data
arrayB  BYTE 10h,20h,30h
.code
mov  esi,OFFSET arrayB
mov  al,[esi]           ; AL = 10h
inc  esi
mov  al,[esi]           ; AL = 20h
inc  esi
mov  al,[esi]           ; AL = 30h
```

```
.data
arrayW  WORD 1000h,2000h,3000h
.code
mov  esi,OFFSET arrayW
mov  ax,[esi]           ; AX = 1000h
add  esi,2
mov  ax,[esi]           ; AX = 2000h
add  esi,2
mov  ax,[esi]           ; AX = 3000h
```

Offset	Value	
10200	1000h	← [esi]
10202	2000h	
10204	3000h	

Indirect Addressing

Example: Adding 32-Bit Integers

```
.data
arrayD DWORD 10000h,20000h,30000h
.code
mov esi,OFFSET arrayD
mov eax,[esi]           ; first number
add esi,4
add eax,[esi]           ; second number
add esi,4
add eax,[esi]           ; third number
```

Offset	Value	
10200	10000h	← [esi]
10204	20000h	← [esi] + 4
10208	30000h	← [esi] + 8

Indexed Operands

An indexed operand adds a constant to a register to generate an effective address.

constant[reg]
[constant + reg]

arrayB[esi]	[arrayB + esi]
arrayD[ebx]	[arrayD + ebx]

```
.data
arrayB BYTE 10h,20h,30h
.code
mov esi,0
mov al,arrayB[esi]           ; AL = 10h
```

Indirect Addressing

Adding Displacements

```
.data
arrayW  WORD 1000h,2000h,3000h
.code
mov  esi,OFFSET arrayW
mov  ax,[esi]           ; AX = 1000h
mov  ax,[esi+2]         ; AX = 2000h
mov  ax,[esi+4]         ; AX = 3000h
```

Using 16-Bit Registers

```
mov  al,arrayB[si]
mov  ax,arrayW[di]
mov  eax,arrayD[bx]
```

Scale Factors in Indexed Operands

Indexed operands must take into account the size of each array element when calculating offsets.

```
.data
arrayD  DWORD 100h, 200h, 300h, 400h
.code
mov  esi, 3 * TYPE arrayD           ; offset of arrayD[3]
mov  eax, arrayD[esi]                ; EAX = 400h
```

```
.data
arrayD  DWORD 1, 2, 3, 4
.code
mov  esi, 3                          ; subscript
mov  eax, arrayD[esi*4]               ; EAX = 4
```

```
mov  esi, 3                          ; subscript
mov  eax, arrayD[esi*TYPE arrayD]    ; EAX = 4
```

Pointers

A variable containing the address of another variable is called a pointer.

```
.data
```

```
arrayB byte 10h,20h,30h,40h
```

```
ptrB dword arrayB          ptrB contains the offset of arrayB
```

```
ptrB dword OFFSET arrayB
```

```
arrayB    BYTE    10h,20h,30h,40h
```

```
arrayW    WORD     1000h,2000h,3000h
```

```
ptrB      DWORD    arrayB          ptrB contains the offset of arrayB
```

```
ptrW      DWORD    arrayW          ptrW contains the offset of arrayW
```

```
ptrB      DWORD    OFFSET arrayB
```

```
ptrW      DWORD    OFFSET arrayW
```

Using the TYPEDEF Operator

The TYPEDEF operator lets you create a user-defined type that has all the status of a built-in type when defining variables. TYPEDEF is ideal for creating pointer variables.

```
PBYTE TYPEDEF PTR BYTE
```

```
.data
```

```
arrayB BYTE 10h,20h,30h,40h
```

```
ptr1    PBYTE ?                ; uninitialized
```

```
ptr2    PBYTE arrayB           ; points to an array
```

Indirect Addressing

Example Program: Pointers

```
; Pointers (Pointers.asm)

; Demonstration of pointers and TYPEDEF.

.386
.model flat,stdcall
.stack 4096
ExitProcess PROTO, dwExitCode:dword

; Create user-defined types.
PBYTE  TYPEDEF PTR BYTE           ; pointer to bytes
PWORD  TYPEDEF PTR WORD           ; pointer to words
PDWORD TYPEDEF PTR DWORD          ; pointer to doublewords

.data
arrayB BYTE 10h,20h,30h
arrayW WORD 1,2,3
arrayD DWORD 4,5,6

; Create some pointer variables.
ptr1 PBYTE arrayB
ptr2 PWORD arrayW
ptr3 PDWORD arrayD
```

```
.code
main PROC

; Use the pointers to access data.
    mov esi,ptr1
    mov al,[esi]                ; 10h
    mov esi,ptr2
    mov ax,[esi]                ; 1
    mov esi,ptr3
    mov eax,[esi]               ; 4

    invoke ExitProcess,0
main ENDP
END main
```

Indirect Addressing

Which type of control transfer should be used?

Unconditional Transfer

Control is always transferred to a new location, ensuring execution continues at the new address.



Conditional Transfer

Control is transferred based on a condition, allowing for branching logic.

Made with  Napkin

JMP Instruction

The JMP instruction causes an unconditional transfer to a destination, identified by a code label that is translated by the assembler into an offset.

```
JMP destination
```

Creating a Loop

```
top:
    .
    .
    jmp top                ; repeat the endless loop
```

LOOP Instruction

The LOOP instruction, formally known as Loop According to ECX Counter, repeats a block of statements a specific number of times. ECX is automatically used as a counter and is decremented each time the loop repeats.

LOOP destination

```
    mov    ax,0
    mov    ecx,5
L1:  inc    ax
     loop  L1
```

Nested Loops

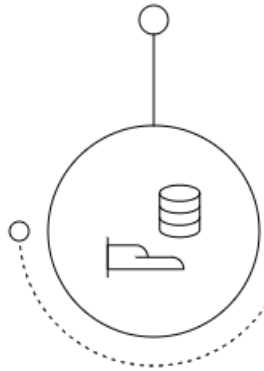
```
.data
count DWORD ?
.code
    mov    ecx,100                ; set outer loop count
L1:      mov    count,ecx          ; save outer loop count
         mov    ecx,20            ; set inner loop count
L2:      .
         .
         loop   L2                ; repeat the inner loop
         mov    ecx,count         ; restore outer loop count
         loop   L1                ; repeat the outer loop
```

Summing an Integer Array

Assembly Language Array Summation

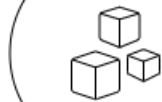
Assign Array Address

Load the array's starting address into a register



Initialize Sum Register

Set the sum register to zero



Initialize Loop Counter

Set the loop counter to the array's length



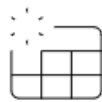
Mark Loop Start

Create a label to mark the beginning of the loop



Add Element to Sum

Add the current array element to the sum



Point to Next Element

Increment the register to point to the next element



Repeat Loop

Use the LOOP instruction to repeat the process



JMP and Loop instructions

```
; Summing an Array (SumArray.asm)
; This program sums an array of words.

.386
.model flat,stdcall
.stack 4096
ExitProcess PROTO, dwExitCode:dword

.data
intarray DWORD 10000h,20000h,30000h,40000h

.code
main proc

        mov  edi,OFFSET intarray      ; 1: EDI = address of intarray
        mov  ecx,LENGTHOF intarray    ; 2: initialize loop counter
        mov  eax,0                     ; 3: sum = 0
L1:      ; 4: mark beginning of loop
        add  eax,[edi]                 ; 5: add an integer
        add  edi,TYPE intarray         ; 6: point to next element
        loop L1                       ; 7: repeat until ECX = 0

        invoke ExitProcess,0

main endp
end main
```

Copying a String

```
; Copying a String (CopyStr.asm)
; This program copies a string.
.386
.model flat,stdcall
.stack 4096
ExitProcess PROTO, dwExitCode:dword

.data
source byte "This is the source string",0
target byte SIZEOF source DUP(0),0

.code
main proc

        mov esi,0                ; index register
        mov ecx,SIZEOF source    ; loop counter
L1:      mov al,source[esi]        ; get a character from source
        mov target[esi],al        ; store it in the target
        inc esi                  ; move to next character
        loop L1                  ; repeat for entire string

        invoke ExitProcess,0

main endp
end main
```

JMP and Loop instructions

4 Data Transfers, Addressing, and Arithmetic 95

4.1 Data Transfer Instructions 96

- 4.1.1 Introduction 96
- 4.1.2 Operand Types 96
- 4.1.3 Direct Memory Operands 96
- 4.1.4 MOV Instruction 98
- 4.1.5 Zero/Sign Extension of Integers 99
- 4.1.6 LAHF and SAHF Instructions 101
- 4.1.7 XCHG Instruction 102
- 4.1.8 Direct-Offset Operands 102
- 4.1.9 Example Program (Moves) 103
- 4.1.10 Section Review 104

4.2 Addition and Subtraction 105

- 4.2.1 INC and DEC Instructions 105
- 4.2.2 ADD Instruction 105
- 4.2.3 SUB Instruction 106
- 4.2.4 NEG Instruction 106
- 4.2.5 Implementing Arithmetic Expressions 106
- 4.2.6 Flags Affected by Addition and Subtraction 107
- 4.2.7 Example Program (*AddSubTest*) 111
- 4.2.8 Section Review 112

4.3 Data-Related Operators and Directives 112

- 4.3.1 OFFSET Operator 112
- 4.3.2 ALIGN Directive 113
- 4.3.3 PTR Operator 114
- 4.3.4 TYPE Operator 115
- 4.3.5 LENGTHOF Operator 116
- 4.3.6 SIZEOF Operator 116
- 4.3.7 LABEL Directive 116
- 4.3.8 Section Review 117

4.4 Indirect Addressing 117

- 4.4.1 Indirect Operands 117
- 4.4.2 Arrays 118
- 4.4.3 Indexed Operands 119
- 4.4.4 Pointers 121
- 4.4.5 Section Review 122

4.5 JMP and LOOP Instructions 123

- 4.5.1 JMP Instruction 123
- 4.5.2 LOOP Instruction 124
- 4.5.3 Displaying an Array in the Visual Studio Debugger 125
- 4.5.4 Summing an Integer Array 126
- 4.5.5 Copying a String 127
- 4.5.6 Section Review 128

4.6 64-Bit Programming 128

- 4.6.1 MOV Instruction 128
- 4.6.2 64-Bit Version of SumArray 130
- 4.6.3 Addition and Subtraction 130
- 4.6.4 Section Review 131

4.7 Chapter Summary 132

4.8 Key Terms 133

- 4.8.1 Terms 133
- 4.8.2 Instructions, Operators, and Directives 133

4.9 Review Questions and Exercises 134

- 4.9.1 Short Answer 134
- 4.9.2 Algorithm Workbench 136

4.10 Programming Exercises 137

reference

<https://www.asmirvine.com/>

<https://intelligentcm.tistory.com/category/Assembly>

Review questions and Exercises
&
Programming exercises

Github upload

Presentation subject